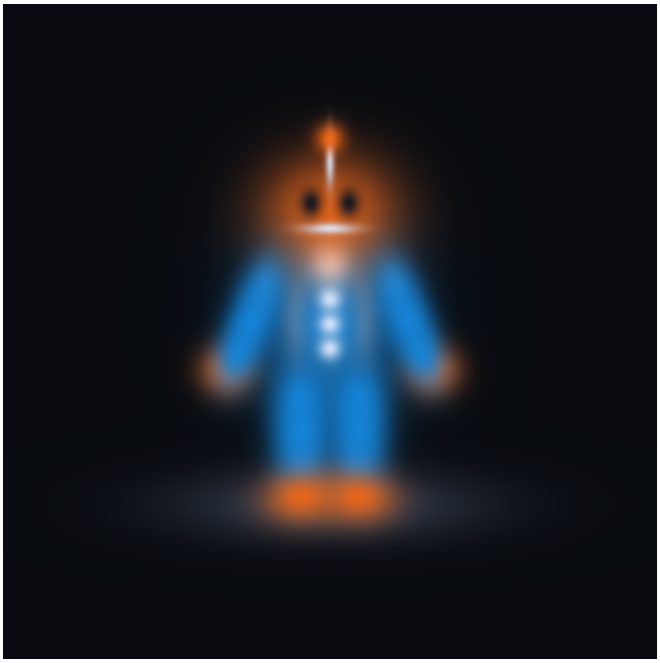
Build a Gaussian renderer you can explain

A one-day workbook for Cody Jung

A small 3D renderer, a real gradient-based training loop, and evidence you can defend in an interview.

This guide assumes you can write Python but are new to computer vision, differentiable rendering, and the mechanics of training. It introduces the mathematics at the point where you need it.



Our supplied 31-Gaussian synthetic target, rendered at 512 x 512. Its soft appearance comes from the representation, not a low-resolution screenshot.

PROJECT GOAL / The end result

Implement five core functions, recover a simple scene from several images, evaluate unused camera views, and export a short camera orbit. Understand every operation between a 3D Gaussian and an image error.

The PDF is the learning path. The companion ZIP supplies the setup and experiment machinery, with separate reference answers. You will not need to transcribe long scripts from a PDF.

Prepared 7 September 2026. Setup links were checked on that date. GPU availability and interface labels may change.

Your route and definition of done

Read each concept, predict a result, implement the relevant function, then run its check. Do not read the reference answer before making an attempt.

Pages	Work	Planning allowance
3-5	Environment, files, essential ML concepts	45-75 min
6-14	Gaussians, camera, projection, pixels, blending	3-5 hours
15-19	RunPod, dataset, training, gradient check	2-3 hours
20-24	Runs, resolution, experiments, presentation	2-3 hours
25-30	Troubleshooting and interview rehearsal	1-2 hours

These are ambitious work estimates, not guaranteed completion times. Training can be quick while debugging takes hours. Reserve the last 90 minutes for explaining your actual implementation.

What you will implement

In core.py: covariance(), project(), pixel_weights(), composite(), and update(). Then reconstruct the bounded-footprint loop in support.py after understanding the supplied version. The latter is a forward-rendering exercise; the dense path remains the training reference.

What is supplied

Camera construction, quaternion conversion, a procedural target scene, file handling, training orchestration, plots, export, and checks. Read support.py and run.py before describing them in an interview. These are teaching scaffolds, not contributions you should claim to have independently invented.

CHECKPOINT / Done means evidence, not just execution

You have passing mathematical checks; a falling image error; before/after/target comparisons from unused cameras; an ordering experiment; one-view versus multi-view results; and a written account of what you reused.

WHEN STUCK / When you get stuck

Spend 10-15 minutes isolating the smallest failure. Use the local hint first. Then inspect only that function in reference/core.py. Rewrite it in your own working file and explain why it works. Record assistance honestly; retyping alone is not proof of understanding.

01 / Set up your laptop

Use your Ryzen CPU for the first checks. Your 24 GB RAM is sufficient for the small scenes here. We will use an NVIDIA GPU remotely for repeated experiments. Do not spend the day configuring your integrated GPU or NPU.

These commands assume Windows PowerShell. Install Python 3.12 from python.org/downloads if the Python launcher cannot find it. Download and extract Gaussian_Workbook_Starter.zip; open the extracted gaussian_workbook folder in VS Code. Open a PowerShell terminal in that folder.

```
py -3.12 --version
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install torch --index-url https://download.pytorch.org/whl/cpu
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

No activation is necessary. Use the explicit interpreter path, so PowerShell execution policy does not block you. In VS Code, choose Python: Select Interpreter, then .venv/Scripts/python.exe.

```
.\.venv\Scripts\python.exe -c "import torch; print(torch.__version__); `
    print(torch.cuda.is_available())"
.\.venv\Scripts\python.exe run.py check --stage covariance
```

CHECKPOINT / Expected result

The first command prints a PyTorch version and False for CUDA. That is correct locally. The second command raises NotImplementedError for TODO 1. This confirms the starter uses YOUR unfinished core.py rather than the reference answers.

Command shorthand in this guide

Later pages use `python run.py ...`. On your laptop, replace `python` with `.\.venv\Scripts\python.exe`. In the RunPod terminal, use `python` as written. Commands are one line each unless explicitly shown otherwise.

WHEN STUCK / If setup fails

Run `Get-Location` and `dir` to confirm you are inside gaussian_workbook and can see requirements.txt. If `py -3.12` fails, install Python 3.12 and reopen the terminal. For package-install choices, consult the official PyTorch selector. Keep the CPU installation local; do not overwrite RunPod's GPU-enabled PyTorch with it.

02 / Know where everything belongs

The ZIP already contains the files below. Work from the gaussian_workbook directory. There is no repository, API key, pretrained weight download, or image dataset required for the main path.

File	Your action
core.py	Implement the five TODO functions. This is your main working file.
support.py	Read the camera, parameter model, renderer wiring, and target-scene helpers. Rebuild the bounded loop after the dense renderer works.
run.py	Use its commands; inspect the training orchestration before the interview.
reference/core.py	Supplied answer key. Open one function at a time if needed.
requirements.txt	Supporting packages. PyTorch is installed separately.
COMMANDS.md	Copy-friendly local and remote commands.
NOTES.md	Record predictions, bugs, decisions, and actual results.
validation/	Author's small reference-run evidence, clearly labeled.
outputs/	Created when you run commands; contains YOUR results.

Function contracts

N means the number of Gaussians. P means the number of pixels, equal to height times width. A tensor is a multidimensional array; its shape describes its axes.

```
import torch
# Most calculations use torch.float32.
# Every participating tensor must be on the same device.
# Mean positions:       [N, 3]
# 3D covariances:       [N, 3, 3]
# Projected positions:  [N, 2]
# 2D covariances:       [N, 2, 2]
# Pixel weights:        [N, P]
# Image:               [height, width, 3]
```

CHECKPOINT / Ownership checkpoint

Find the import in run.py that chooses core.py. Without --reference, your functions execute. With --reference, the supplied answer key executes. Do not leave that flag in a command you present as a run of your own implementation.

INTERVIEW / Likely question

"Which parts did you write?" Be able to name functions and explain their inputs and outputs. Distinguish implementing an established algorithm, adapting supplied code, and inventing a new method.

03 / The minimum training theory

A parameter is a number the system can adjust. Here, parameters describe a scene. Training repeatedly changes those numbers so rendered images better match target images.

A loss is one number that measures disagreement. We use mean squared error (MSE): subtract target RGB values from predicted RGB values, square each difference, then average over pixels and color channels. Lower is better. Colors are floats in [0, 1], not integers in [0, 255].

A gradient tells us how the loss changes for a small change in a parameter. Gradient descent subtracts a small multiple of that gradient. The learning rate controls the update size. Adam is an optimizer that adapts update sizes using recent gradients; PyTorch supplies it.

```
x = torch.tensor(2.0, requires_grad=True)
loss = (x - 5.0).square()
loss.backward()
print(x.grad)                      # tensor(-6.)
with torch.no_grad():
    x -= 0.1 * x.grad              # x becomes 2.6
```

The negative gradient means increasing x locally reduces the loss. `backward()` computes derivatives by following the recorded chain of operations. It does not itself change x. The update changes x.

Reading: PyTorch autograd basics

Why no neural network is required

A neural network is one way to parameterize a function. Our function is a renderer, and its parameters are explicit scene attributes. We can differentiate the rendering calculations and optimize those attributes directly.

CHECKPOINT / Say this without reading

"The scene is the parameter set. Rendering predicts an image. The loss compares it with a target. Backpropagation computes derivatives. The optimizer changes the scene."

WHEN STUCK / Two common traps

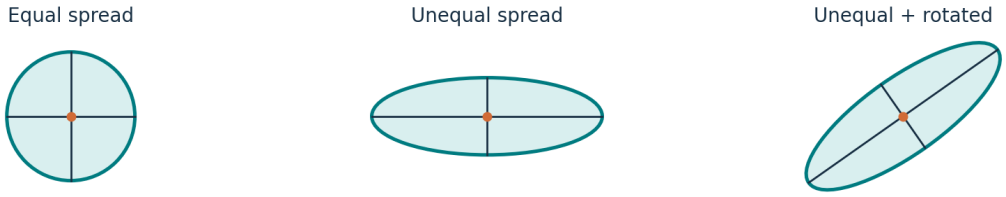
Gradients accumulate unless cleared between updates. Also, converting a differentiable tensor to NumPy, calling detach(), or using no_grad() inside the training render breaks the connection to the parameters. Those operations are appropriate for saving images or generating fixed targets.

04 / A Gaussian is a spatial influence function

A 3D Gaussian assigns a smoothly decreasing value to locations around its center. In this renderer it also carries a color and opacity. It is not a solid object with a sharp boundary.

Parameter	Meaning	Working shape
Mean / position	Center in world coordinates	[N, 3]
Scale	Spread along three local axes; positive	[N, 3]
Rotation	Orientation of those axes	[N, 4] quaternion
Opacity	Peak opacity before compositing	[N]
Color	Fixed red, green, blue values	[N, 3]

A spherical, or isotropic, Gaussian has equal spread in all directions. An anisotropic Gaussian has different spreads. An elongated Gaussian can cover an elongated feature with fewer primitives than many small spherical ones, although the quality/cost tradeoff depends on fitting and scene structure.



Two-dimensional shapes showing the same spread and orientation ideas used in 3D.

Learnable numbers versus physical values

The supplied Scene class stores unconstrained numbers and converts them when rendering:

```
scales = log_scales.exp()           # strictly positive
opacity = opacity_logits.sigmoid()  # between 0 and 1
colors = color_logits.sigmoid()     # between 0 and 1
```

A logit is an unconstrained input to the sigmoid function. A quaternion is a four-number representation of a rotation; our order is (w, x, y, z). Normalize it before converting to a rotation matrix. The supplied quaternion_matrix() does that. Identity rotation is [1, 0, 0, 0], not all zeros.

CHECKPOINT / Inspect support.py
In teacher_scene(), identify the body, eyes, and limbs. In Scene.physical(), identify what is stored versus returned. There are 31 Gaussians in the default target.

INTERVIEW / Why not just points?
A point alone has no spread to cover nearby pixels. A Gaussian gives a controllable smooth footprint and gradients as it moves. It does not eliminate all visibility discontinuities or guarantee easy training.

04 continued / Build the covariance

The covariance matrix describes spread and orientation together. It is 3 x 3 in world space. Its eigenvectors point along the ellipsoid's principal axes; its eigenvalues are squared scales along those axes.

Start with a sphere of unit spread. Scale its axes, then rotate them. If S is a diagonal matrix of scales and R is the rotation, the resulting covariance is:

```
Sigma = R S S^T R^T
      = R diag(sx^2, sy^2, sz^2) R^T
```

The superscript T means transpose: swap rows and columns. Multiplying by R rotates an axis into world space. The transpose on the other side makes the covariance transform consistently in both of its directions.

Implement TODO 1 in core.py

Input scales is [N, 3]; rotations is [N, 3, 3]. Return [N, 3, 3]. Use torch.diag_embed to construct a diagonal matrix for every Gaussian. `@` is matrix multiplication; `*` is elementwise multiplication.

WHEN STUCK / Implementation hint

Build D = torch.diag_embed(scales.square()). Then multiply R, D, and R.transpose(-1, -2), in that order. Do not use R.T on a three-dimensional batch; transpose only the final two axes.

```
Scale (2, 1, 1), identity rotation: diagonal = (4, 1, 1)
Rotate 90 degrees around z:       diagonal = (1, 4, 1)
```

```
python run.py check --stage covariance
```

CHECKPOINT / Expected behavior

The test passes the rotated-axis example and checks positive eigenvalues. Equal scales should give the same covariance for every valid rotation. Try that additional check yourself.

INTERVIEW / Why not learn nine arbitrary covariance entries?

An arbitrary matrix may be asymmetric or have negative variance in some direction. Scale plus rotation constructs a valid positive-definite covariance when scales are positive. Other valid parameterizations, such as a Cholesky factor, also exist. Do not claim this is the only possible choice.

If matrices still feel like symbols, watch 3Blue1Brown: Linear transformations and matrices before continuing. Focus on how a matrix moves basis directions.

05 / Put the scene in the camera's coordinates

World coordinates describe the shared scene. Camera coordinates describe that same scene relative to one camera. Changing the camera does not require changing the stored scene parameters.
We use a single convention throughout: camera x is right, y is up, z is forward. Visible Gaussian centers have z > 0.1. Image coordinates u move right and v move down.
The camera has an eye position and an orthonormal matrix W. Each row of W is one camera axis expressed in world coordinates. A dot product with that row extracts a coordinate along that axis.

```
Column-vector equation: camera_mean = W (world_mean - eye)
Row-stored PyTorch:     cam = (means - eye) @ W.T
Camera covariance:     cov_cam = W @ cov_world @ W.T
```

Subtract the camera position first, then change axes. Translation moves the center but does not change the spread around that center, so it does not appear in the covariance equation.

Read the supplied camera() helper

It points the forward axis from eye toward the object. Cross products construct perpendicular right and up axes; normalization gives unit-length axes. The world-up hint must not be parallel to the viewing direction. Our fixed orbit avoids that case.

CHECKPOINT / Numeric check
With W = identity and eye = (1, 0, 0), a world point (1, 0, 2) becomes (0, 0, 2). It is directly in front of the camera. Moving the eye right makes a fixed world point move left in camera coordinates.

WHEN STUCK / If the object is mirrored or upside down
Print one point rather than changing signs randomly. Confirm +x moves u right and +y moves v up. Do not combine camera matrices copied from OpenGL or COLMAP with this convention without converting them.

INTERVIEW / Expected questions
"Why does translation not affect covariance?" Because spread is measured relative to the mean. "What is W inverse?" W.T, for this orthonormal camera rotation. "What are camera intrinsics?" Pixel focal lengths and image-center offsets; they describe projection inside the camera, while its pose describes location and orientation.

06 / Perspective projection and its Jacobian

For a camera-space point (x, y, z), compute its image location:

```
u = fx * x / z + cx
v = cy - fy * y / z
```

fx and fy are focal lengths measured in pixels. cx and cy are the image center. The minus sign in v converts camera-up to image-down. Our square images use fx = fy = 0.9 * size and cx = cy = (size - 1) / 2.

Dividing by z makes distant objects look smaller. A point on the viewing axis, x = y = 0, projects to the image center regardless of depth.

What the Jacobian means

The Jacobian J records how u and v respond to small changes in x, y, and z. Each row corresponds to an output coordinate; each column corresponds to an input coordinate. It is a 2 x 3 matrix.

```
J = [ fx/z      0      -fx*x/z^2 ]
    [   0     -fy/z     fy*y/z^2 ]

small_screen_change approximately = J @ small_3D_change
```

The x derivative of fx*x/z is fx/z. Its z derivative is -fx*x/z^2: at an off-center point, increasing depth pulls the projected point toward the center. The y row follows the same calculation with the image-coordinate sign.

CHECKPOINT / Work one example

Set fx = fy = 100, cx = cy = 32, and (x, y, z) = (1, 0, 2). The point lands at (82, 32). J's first row is (50, 0, -25). Increasing x by 0.01 moves u by approximately +0.5 pixels; increasing z by 0.01 moves u by approximately -0.25 pixels.

INTERVIEW / Why only approximately?

Perspective divides by a changing depth, so it is nonlinear. J describes the local behavior around one mean. Large Gaussians, substantial depth spread, or locations near the camera can make that approximation inaccurate. We do not claim that exact perspective projection preserves a Gaussian distribution.

06 continued / From an ellipsoid to an ellipse

A covariance tells us how nearby 3D offsets are distributed. The camera rotation transforms those offsets first. The projection Jacobian then approximates how they move on the screen.

```
cov_cam = W @ cov_world @ W.T
cov2 = J @ cov_cam @ J.T
Shapes: [2,3] @ [3,3] @ [3,2] = [2,2]
```

For a linear transformation A, covariance transforms as A Sigma A^T. This follows by writing each transformed offset as A d and averaging (A d)(A d)^T. Perspective is not linear, so we use its local linear approximation J.

Implement TODO 2 in core.py

Calculate camera means and covariance, then projected means and J, then cov2. Return uv, cov2, and camera depth z. This function receives only centers already in front of the near plane.

WHEN STUCK / Build J without breaking gradients

Use torch.zeros_like(z), torch.stack(), and reshape(-1, 2, 3). Avoid torch.tensor([...]) around existing differentiable tensors: that can copy values into a new tensor disconnected from their computation history.

The renderer also adds blur^2 times the 2 x 2 identity matrix. blur is a small pixel-space standard deviation. This keeps very small ellipses numerically manageable and gives a minimum screen footprint. Our rule is 0.3 * size / 256 pixels. It is a deliberate simplification, not a full antialiasing solution.

```
python run.py check --stage projection
```

CHECKPOINT / Expected behavior

The center is correct. Doubling distance on the viewing axis halves the projected standard deviation and quarters the covariance, before blur is added. The test also checks an off-axis point so a missing depth-derivative term does not go unnoticed.

INTERVIEW / Why an ellipse?

For a positive-definite 2 x 2 covariance, points with equal quadratic distance form ellipses. Its two eigenvectors give screen-axis directions; the square roots of its eigenvalues give spreads. The ellipse is a constant-influence contour, not a sharp physical boundary.

07 / Compute each splat's pixel influence

Flatten the image into P pixel coordinates. For one Gaussian, subtract its projected center from each pixel to get the 2D offset d.

```
d = pixel - projected_mean
q = d^T inverse(cov2) d
g = exp(-0.5 * q)
alpha = min(0.99, opacity * g)
```

q measures squared distance after accounting for the ellipse's spread and orientation. Moving two pixels along a broad axis may matter less than moving one pixel along a narrow axis. At the center q = 0 and g = 1; influence decreases smoothly outward.

Implement TODO 3 in core.py

Return weights g with shape [N, P]. The caller multiplies by opacity afterward. We do not divide by a Gaussian probability-density normalization constant: this kernel has peak 1, and opacity sets its peak contribution.

WHEN STUCK / Shapes and a useful operation

`d = pixels[None, :, :] - uv[:, None, :]` gives [N, P, 2]. For this small 2 x 2 problem, torch.linalg.inv(cov2) is sufficient. You can evaluate the quadratic form with `torch.einsum('npi,nij,npj->np', d, inv, d)`. n selects a Gaussian, p a pixel, and i/j the two coordinates being summed.

```
cov2 = diag(4, 1)
Offset (0, 0): q = 0, g = 1
Offset (2, 0): q = 1, g = exp(-0.5), about 0.607
Offset (0, 1): q = 1, g = exp(-0.5), about 0.607
```

```
python run.py check --stage weights
```

CHECKPOINT / Expected behavior

Those values pass. The kernel is symmetric around its mean. A narrow Gaussian has a smaller footprint, not a larger peak. If weights exceed 1 by a meaningful amount, inspect q and covariance validity.

INTERVIEW / Is g a probability?

It is a nonnegative influence weight. The image is produced through opacity-weighted compositing; we are not sampling a normalized probability distribution over pixels.

08 / Sort and composite

Sort Gaussians by increasing camera-space center depth: near first. For each pixel, accumulate visible color and track how much of what lies behind remains visible.
That remaining fraction is transmittance T. It starts at 1. After a splat with alpha = 0.5, it becomes 0.5. After another alpha = 0.5, it becomes 0.25.

```
Start: C = (0, 0, 0), T = 1
For each Gaussian, near to far:
    C = C + T * alpha * color
    T = T * (1 - alpha)
Finish: C = C + T * background
```

Implement TODO 4 in core.py

Inputs alpha [N, P] and colors [N, 3] are already sorted. Return [P, 3]. The transmittance used for a splat must include only earlier splats: it is an exclusive product.

WHEN STUCK / Vectorized solution

Prepend a row of ones to (1 - alpha), then take torch.cumprod along the Gaussian axis. All rows except the last give transmittance before each splat. The last row is the fraction of background remaining. Multiply T * alpha, then sum weighted colors using matrix multiplication.

A hand calculation worth memorizing

Red in front and blue behind, each alpha = 0.5, over black: final RGB = (0.5, 0, 0.25). Reversing the order gives (0.25, 0, 0.5). With a white background, the first result instead becomes (0.75, 0.25, 0.5).

```
python run.py check --stage blend
```

CHECKPOINT / Expected behavior

Alpha zero leaves the pixel unchanged. A nearly opaque foreground splat nearly hides the background. Including the background term matters whenever total coverage is incomplete.

INTERVIEW / Is center-depth sorting exact?

No. Extended Gaussians overlap in 3D, so a single ordering by their centers is an approximation to visibility. Sorting changes discretely as depths cross; ordinary autograd differentiates values within the selected ordering, not the integer permutation itself.

09 / Read the complete forward pass

The supplied render() in support.py connects your functions. Trace one call in this order:

Order	Operation	Why it exists
1	Convert stored parameters to physical values	Positive scales, bounded colors/opacity
2	Reject centers at z <= 0.1	Avoid division by zero and behind-camera centers
3	Build 3D covariance	Encode shape and orientation
4	Project mean and covariance	Find the screen ellipse
5	Sort by center depth	Establish compositing order
6	Evaluate pixel weights and alpha	Determine local influence
7	Composite and add background	Form the final RGB image

Near-plane rejection is a coarse center-based rule. It does not correctly clip an ellipsoid crossing the near plane. Keep the toy object comfortably in front of each camera.

```
python run.py check
python run.py make --size 64 --out outputs/smoke
```

The first command includes a gradient test; it does not require TODO 5 yet. The second creates target images with your core renderer. Open outputs/smoke/target_00.png. You should see a front view of a soft blue-and-orange robot.

CHECKPOINT / Before moving on

Inspect front and side images. Confirm no NaNs, mirrored eyes, streaks, or giant ellipses. Compare your image against the supplied reference preview if something looks wrong. A plausible image alone is insufficient: the independent numeric checks must pass too.

WHEN STUCK / Follow a broken pixel backward

Wrong color? Check the blend and ordering. Wrong footprint? Check cov2, J, and scale squaring. Wrong position? Check the camera transform and projection. Entirely blank? Count surviving centers and inspect depth before altering opacity.

INTERVIEW / Why keep the first version dense?

A dense renderer evaluates all Gaussian-pixel pairs, making the mathematics easy to inspect and the gradient path straightforward. It is acceptable for this tiny scene. It is not the fast rasterizer from the paper and does not scale to millions of Gaussians.

10 / Avoid touching the whole image

After the dense renderer works, study bounded_forward() in support.py. Reconstruct this loop yourself using the steps below. Keep the working version available for comparison.

For a cutoff at q <= 9, the influence boundary is a three-standard-deviation ellipse. A tight axis-aligned bounding rectangle has half-width 3 * sqrt(cov2[0,0]) and half-height 3 * sqrt(cov2[1,1]). These formulas include the effects of ellipse rotation; the diagonal entries describe spread along the screen axes.

1. Subtract/add those extents from the projected center.
2. Round the lower coordinates down and upper coordinates up.
3. Clip the rectangle to the image; skip an empty rectangle.
4. Construct pixels only inside that rectangle.
5. Evaluate q and discard contributions with q > 9.
6. Update C and T only in that pixel patch, in depth order.

WHEN STUCK / What is actually being approximated?

At q = 9, g = exp(-4.5), about 0.011. Contributions beyond the cutoff are small individually but not zero, and many omitted contributions can accumulate. There is no universal guarantee that the resulting image difference is negligible.

```
python run.py check
```

CHECKPOINT / Expected behavior

The final check compares the bounded renderer with a dense renderer using the SAME cutoff. They should agree numerically. A comparison against the uncut dense renderer measures truncation error instead, so it should not be required to match exactly.

Keep the training path separate

Our bounded implementation uses Python integer boxes and in-place image patches under no_grad(). It is forward-only. Training uses the dense functional implementation. Describe that division explicitly. Simply adding a mask to a full [N, P] array does not avoid computing or allocating that array.

INTERVIEW / What makes real implementations fast?

Spatial binning into tiles, compact candidate lists, parallel GPU work, early exit when T is tiny, and a specialized backward pass. This is substantial algorithm and systems work, not a feature you can claim because you used a GPU.

11 / Introduce RunPod here

Your first render and numeric checks now work. Move to a rented NVIDIA GPU before repeated training. You can keep editing from your laptop; RunPod is where the Python process executes.

Create one Pod

Open RunPod's console. Add payment credit, then choose Pods and Deploy (or + New > Pod). Choose one available NVIDIA GPU, On-Demand pricing, and an official RunPod PyTorch template with Jupyter enabled. Name it gaussian-workbook. Review the displayed GPU and storage costs before deployment. The official guide covers both current interface variants. RunPod deployment guide

For this small experiment, use a compatible GPU with at least 16 GB dedicated memory as a comfortable starting choice. A premium H100 is optional. Select an image explicitly compatible with your selected GPU, especially for recent RTX PRO hardware; avoid an old CUDA image copied from a tutorial.

Choose storage deliberately

Use about 20 GB container storage, or retain a larger template minimum, and 20 GB workspace storage for this small project. We recommend a network volume if available with your chosen GPU; attach it during creation. Otherwise use a volume disk and download checkpoints before termination. Our scene does not need a large dataset disk.

A volume disk generally preserves /workspace across stop/start but is deleted when the Pod is terminated. A network volume survives Pod termination and has its own storage charges. Confirm the actual mount path in the chosen template. RunPod storage types

CHECKPOINT / Access checkpoint

Wait for Running. Choose Connect > Jupyter Lab under HTTP Services, usually port 8888. In Jupyter, open File > New > Terminal. Official PyTorch templates include Jupyter; use the template's stated authentication. Connection instructions

WHEN STUCK / If access fails

Check Pod logs and template startup status first. If the GPU/template combination is incompatible, choose a supported image rather than installing arbitrary CUDA versions by trial and error. No RunPod API key or public custom web server is required for this workflow.

11 continued / Transfer and verify

On your laptop, compress your edited core.py, support.py, run.py, requirements.txt, and reference folder into project.zip. Do not include .venv; Windows packages cannot be reused in a Linux container. In Jupyter's file browser, navigate to /workspace and upload project.zip. Open a terminal:

```
cd /workspace
mkdir -p gaussian_workbook
python -m zipfile -e project.zip gaussian_workbook
cd gaussian_workbook
ls
python -m pip install -r requirements.txt
nvidia-smi
python -c "import torch; print(torch.__version__); print(torch.cuda.is_available()); \
    print(torch.cuda.get_device_name(0))"
```

If the ZIP contains an outer directory, enter that directory until ls shows run.py. Keep RunPod's existing PyTorch installation. The requirements file deliberately omits torch.

```
python run.py check --device cuda
python -m pip freeze > environment-runpod.txt
```

CHECKPOINT / Expected result

nvidia-smi reports the rented GPU. PyTorch prints True and the same GPU model. All checks pass on CUDA. Save the template/image name and GPU model in NOTES.md. You now have a recorded environment rather than a guessed version combination.

Keep long runs alive

If tmux is available, run `tmux new -s gs` before training. If missing on the official Ubuntu template, install it with `apt-get update` then `apt-get install -y tmux`. Detach with Ctrl+B, then D. Reconnect with `tmux attach -t gs`. This survives browser disconnection, not Pod shutdown.

Retrieve results

At the end, run `python -m zipfile -c results.zip outputs core.py support.py run.py NOTES.md environment-runpod.txt`. In Jupyter, right-click results.zip and download it. Open the downloaded archive locally before deleting remote resources. For larger transfers, use RunPod's transfer guide.

WHEN STUCK / GPU unavailable in PyTorch

If nvidia-smi works but torch.cuda.is_available() is False, check whether a CPU-only torch wheel was installed. Restore a compatible official PyTorch template rather than running the laptop CPU-install command remotely.

12 / Define the learning problem precisely

Our target is a stylized robot made of 31 Gaussians. A supplied helper specifies its parameters. Your renderer generates 12 fixed target views around it. Eight are used for training; four are reserved for evaluation.

```
python run.py make --device cuda --size 512 --out outputs/data
```

Open several target images before training. The dataset.npz contains target images, camera angles, split indices, and an initial scene. Training at smaller sizes area-downsamples these fixed 512 x 512 targets; it does not regenerate targets from the current learned scene.

What the initialization gives you

The starting scene uses the correct number of Gaussians and perturbed target positions, scales, rotations, and colors. Opacity starts at 0.8. This is a helpful starting point. It makes the first optimization experiment reliable enough to debug in a day.

The training loss compares images only. It does not compare learned parameters with target parameters. Nevertheless, the starting point contains substantial geometric information. Call this "synthetic scene recovery from a perturbed initialization," not "3D reconstruction from arbitrary photographs."

CHECKPOINT / Data separation

Train indices are 0, 2, 3, 5, 6, 8, 9, 11. Held-out indices are 1, 4, 7, 10. The same initial scene and held-out views are reused in the one-view and multi-view experiment. Do not choose different initializations for those two runs.

WHEN STUCK / Why synthetic targets?

They remove file-format, calibration, and scene-capture problems while you learn rendering and optimization. Because targets and predictions use the same renderer, shared bugs can cancel. That is why the hand calculations, off-axis projection check, and gradient check are required independently.

INTERVIEW / What would make the task harder?

Random or sparse-point initialization; independently rendered images; unknown cameras; complex reflectance; or unknown Gaussian count. Name the difficulty before proposing it as the next step. A failed harder initialization is informative evidence, not something to hide.

12 continued / Write TODO 5

A training update renders one selected camera, measures error, computes gradients, and changes the scene. This is the fifth function in core.py.

```
def update(prediction, target, optimizer):
    # TODO: clear old gradients
    # TODO: compute a scalar mean squared image error
    # TODO: compute gradients
    # TODO: update the parameters
    # TODO: return the loss tensor
```

WHEN STUCK / Exact operations, in order

optimizer.zero_grad(set_to_none=True); loss = (prediction - target).square().mean(); loss.backward(); optimizer.step(); return loss. The supplied runner has already rendered prediction with gradients enabled. Check that loss is finite before updating if you expand this function.

What the supplied runner does

It selects one training view per update, cycling through the chosen indices. It supplies an Adam optimizer, logs errors, saves checkpoints, and evaluates the unused views. Read train() in run.py now. Moving this orchestration into a function does not change how learning works.

Learn in two stages

`--learn basic` adjusts positions and colors while holding scale, rotation, and opacity fixed at their initial values. This isolates easier failure cases; it cannot perfectly match a target whose shapes also differ.

`--learn all` adjusts all five parameter groups. The supplied learning rates are starting values, not universal settings: positions 0.003, color logits 0.03, log-scales 0.006, quaternions 0.003, opacity logits 0.01. `--lr-factor 0.5` halves all of them.

CHECKPOINT / Before a long run

For each trainable parameter, confirm a gradient tensor exists and contains finite numbers. A zero gradient for some elements can be legitimate; no gradient for an entire expected parameter group may indicate a disconnected calculation. The runner checks missing and non-finite gradients.

INTERVIEW / Why different learning rates?

Position, scale, rotation, and color have different units and sensitivities. The same numerical step can have very different visual consequences. Increasing the learning rate can speed progress, but it can also overshoot, destabilize scales, or make splats disappear from useful views.

13 / Prove the training signal is connected

A small loss is not proof that the derivatives are correct. Check one derivative independently with a finite difference: change a parameter slightly and observe how the loss changes.

```
finite_difference = (loss(theta + epsilon)
                   - loss(theta - epsilon)) / (2 * epsilon)
```

Compare this number with the gradient from backward(). The supplied check uses one Gaussian, a position coordinate, float64 arithmetic, and epsilon = 0.00001. It avoids a depth-order swap or clipping boundary.

```
python run.py check --stage gradient
python run.py check --device cuda --stage gradient
```

CHECKPOINT / Expected result

Both versions should pass. In the reference validation, autograd and the finite difference agreed at approximately 0.00000242415. Your environment can differ slightly. Inspect the test in run.py and try one color or scale coordinate as a follow-up.

Follow one gradient path aloud

Image loss changes with the predicted pixel color. Pixel color changes with compositing weights. Weights change with Gaussian influence. Influence changes with projected center and covariance. Those change with the world-space position, scale, and rotation. The chain rule combines these local sensitivities.

Opacity and color also connect directly to compositing. If foreground transmittance becomes very small, the loss is less sensitive to hidden Gaussians. More camera views can expose them.

WHEN STUCK / If the finite difference disagrees

Use float64, a nonzero derivative, and a moderate epsilon. Too small can cause roundoff; too large no longer measures a local change. Restore the original parameter afterward. Check whether your perturbation crossed a cutoff, near plane, alpha clamp, or sorting boundary.

INTERVIEW / Are all operations differentiable?

Many arithmetic operations are. Sorting, hard culling, integer bounding boxes, and threshold masks make the full system piecewise-defined. Within a fixed ordering and selection, gradients can still be useful. Our dense path avoids integer footprint construction during training; it does not make visibility globally smooth.

14 / Get the first successful fit

All commands below run from the project directory on RunPod. Use your functions: omit --reference. Always use different output folders for different experiments.

First, a short diagnostic run

```
python run.py train --device cuda --size 128 --steps 200 --learn basic --out outputs/basic
```

Open outputs/basic/loss.png, before.png, target.png, and heldout_01.png. Loss should usually improve overall, but consecutive points use different cameras and need not decrease monotonically. Shape errors can remain because this run freezes shape parameters.

Then, optimize the full parameter set

```
python run.py train --device cuda --size 256 --steps 1200 --learn all --out outputs/main
```

This starts from the original initialization, not from the basic run. Inspect metrics.json and heldout images. If the loss explodes or turns non-finite, stop and debug; do not wait for all 1,200 updates.

Refine at 512 after measuring runtime

```
python run.py train --device cuda --size 512 --steps 400 --resume outputs/main/checkpoint.pt \
    --lr-factor 0.25 --out outputs/refine
```

Here --resume loads model parameters but deliberately starts fresh Adam state with a smaller learning rate. It is a refinement run, not an exact continuation of the original optimizer trajectory. config.json records the command settings.

CHECKPOINT / Success criterion

Mean held-out error decreases from its initial value, the object improves from multiple unused viewpoints, and the images stay finite. No universal loss threshold proves correctness. A mostly empty background can make global MSE look better than object quality suggests.

WHEN STUCK / If nothing improves after roughly 100 updates

Return to the tiny checks. Confirm targets are fixed, prediction requires gradients, trainable parameters change after step(), and cameras match. Try --learn basic to isolate the issue. More GPU time does not fix incorrect equations.

15 / Choose resolution using a measurement

Resolution determines how finely we sample the projected scene. It does not determine the number of 3D parameters. Moving from 256 to 512 doubles each side and quadruples the pixels. Higher-resolution targets can constrain smaller features, if the scene has enough Gaussians to represent them. A sparse collection of smooth splats stays smooth at high resolution. Training at 512 is valid; it is not required for the first successful optimization.

```
python run.py bench --device cuda --size 128 --steps 100
python run.py bench --device cuda --size 256 --steps 100
python run.py bench --device cuda --size 512 --steps 100
```

The benchmark uses the 31-Gaussian scene, includes backward and optimizer work, performs warm-up, and waits for the GPU before timing. It reports seconds per update and peak allocated tensor memory. Repeat with your real Gaussian count if you change the scene. Why GPU timing needs care

```
estimated run minutes = seconds_per_update * updates / 60
Examples for 2,000 updates:
0.02 seconds/update -> 0.67 minutes
0.10 seconds/update -> 3.33 minutes
0.50 seconds/update -> 16.67 minutes
```

CHECKPOINT / Choose deliberately

If a 512 run fits comfortably within your experiment budget, use it. Otherwise train at 256 and reserve 512 for refinement and final renders. Record actual measurements rather than a GPU's advertised peak compute rate.

Memory is not just the parameter count

At 512, 31 Gaussians create about 8.1 million Gaussian-pixel pairs. One float32 array of that shape occupies about 32.5 MB. Offsets, weights, transmittance, and saved values for backward require several arrays. Raising the count to 1,000 makes a single such array about 1.05 GB; total training memory is much larger.

WHEN STUCK / If you run out of memory

Use one view per update (already the default), lower the Gaussian count or resolution, and avoid saving loss tensors with computation graphs. The runner saves loss.item()-equivalent floats. Chunking or tile-based training is a separate engineering task, not a one-line mask fix.

16 / Run experiments that answer questions

Use the same dataset and initialization. Write your prediction before running each comparison. An experiment supports only the conditions it actually tests.

A / Why does order matter?

```
python run.py ablate --device cuda --size 256 --checkpoint outputs/main/checkpoint.pt --out \
    outputs/ablation
```

Compare sorted.png with reversed.png. Explain a changed overlap using the red/blue hand calculation. The command also exports isotropic_without_retraining.png and bounded_3sigma.png.

B / What does anisotropy contribute here?

The isotropic edit replaces each Gaussian's scales with their geometric mean, preserving the product of scales and therefore ellipsoid volume. Positions, color, and opacity stay fixed. This tests damaging a fitted representation; it does not compare optimally trained isotropic versus anisotropic models.

C / Does one view constrain the whole scene?

```
python run.py train --device cuda --size 256 --steps 1200 --views one --out outputs/one
```

Compare outputs/one with outputs/main: the same initialization, 1,200 updates, resolution, and optimizer settings; only the set of training cameras differs. Both use the same four held-out views. One-view training repeatedly sees view 0; multi-view cycles through eight views.

CHECKPOINT / What to record

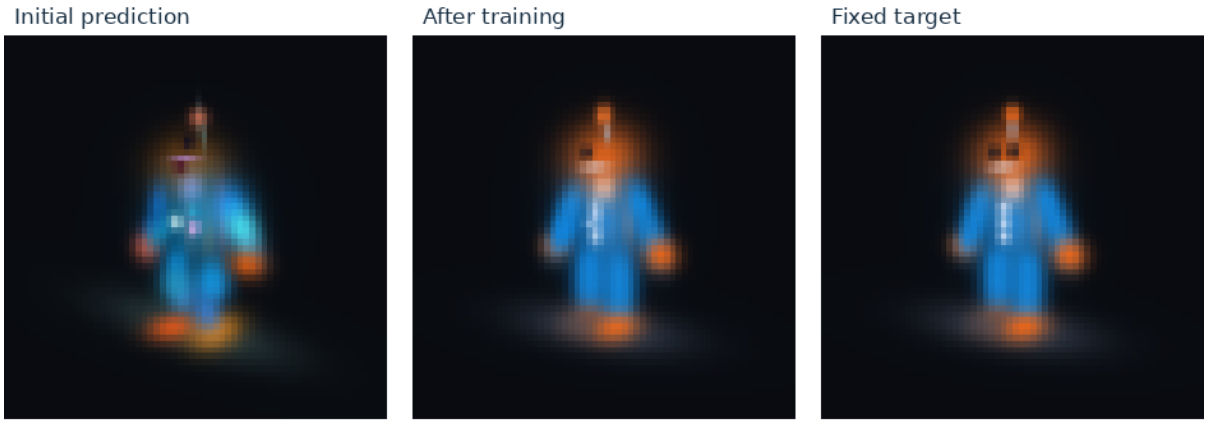
For both runs: training MSE, held-out MSE, one common held-out image, and update count. Compare global error and an object crop visually. One-view training may fit its one image better while doing worse elsewhere. If the gap is small, report that; our helpful initialization already contains much geometry.

INTERVIEW / What is the confound?

Equal update count means equal image evaluations, but unequal distinct information. A second question would compare equal visits per training image, which requires more multi-view updates. State which budget you controlled. Held-out views of the same object test view generalization, not generalization to entirely new objects.

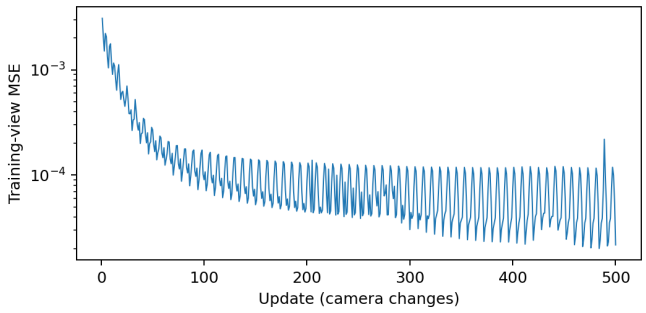
A tested example, not a promised outcome

The supplied reference answers were executed while preparing this guide. This example uses 31 Gaussians, 128 x 128 synthetic target images downsampled to 64 x 64 for training, all parameter groups, and 500 updates. It is a small CPU validation, not the recommended final presentation resolution.



One held-out camera: initial prediction, after multi-view training, and fixed target. Images are enlarged for inspection.

Measurement	Reference result
Initial mean held-out MSE	0.002476
Final mean held-out MSE, multi-view	0.0001243
Held-out error reduction	Approximately 95%
Multi-view loop time	Approximately 5.8 seconds, this environment's CPU



Training loss can fluctuate because successive updates use different camera views.

CHECKPOINT / What this validation establishes

The supplied reference path runs, its numeric checks pass, and a controlled optimization improves held-out image agreement. It does not validate RunPod deployment, your unfinished core.py, full-scene reconstruction, or photorealism. GPU timings must be measured on your rented machine.

The ZIP includes metrics and images under validation/. Those are the guide author's example results. Keep your own outputs separate and present your own measurements.

17 / Package a clear demonstration

Choose the checkpoint based on held-out images and errors. Refinement is not automatically better just because it used more pixels. Compare both checkpoints at the same evaluation resolution when making a quantitative claim.

```
python run.py export --device cuda --size 512 --frames 36 --checkpoint \
    outputs/refine/checkpoint.pt --out outputs/demo
```

If you did not run refinement, substitute outputs/main/checkpoint.pt. The command writes 36 PNGs and orbit.gif, a 3.6-second loop at ten frames per second. The GIF is an offline export, not evidence of real-time rendering. PNGs preserve color more faithfully than a palette-limited GIF.
For a fair final comparison, evaluate both checkpoints at 512 using the eval commands in COMMANDS.md. Compare their evaluation.json files and the same held-out images.

Assemble a three-minute walkthrough

First show the target, initial render, and final held-out render. Explain the input assumptions. Then show the camera orbit and draw the pipeline. Finally show one experiment and discuss its result and limitations.
You do not need a web app. Keep images, the GIF, loss plot, metrics, and the code ready locally. A live change to one Gaussian is a useful bonus only after the saved demo is reliable.

CHECKPOINT / Evidence folder
Keep config.json, metrics.json, loss.csv, loss.png, checkpoint.pt, held-out images, demo/orbit.gif, environment-runpod.txt, and NOTES.md. Record the GPU, resolution, Gaussian count, update count, initialization, and code source used.

Finish the remote session

Download and open results.zip locally. Then stop the Pod to release compute. If you no longer need it, terminate it after confirming the backup. Volume-disk data is deleted on termination; network volumes and their charges remain until separately removed. Verify resource status in the console.
Manage Pods

INTERVIEW / What did you learn beyond making it run?
Answer with a real example from NOTES.md: a sign error you isolated, a gradient you checked, an initialization that failed, or a tradeoff you measured. Do not invent a debugging story or attribute the supplied reference's results to your own implementation.

18 / When something breaks

Symptom	First thing to inspect	Next action
Empty image	Number of centers with z > 0.1	Print camera-space positions; check camera axes and eye
Mirrored / upside-down object	Signs in u and v; camera convention	Test one point moved in +x and +y
Giant ellipse	Tiny positive depth, huge scale	Inspect z, exp(log_scale), and J; keep scene away from near plane
Ellipse rotates incorrectly	Matrix multiplication / quaternion order	Run the 90-degree covariance example
Color is too dark	Exclusive T and background term	Recalculate the two-splat example
Loss has no gradient	detach, NumPy, tensor reconstruction, no_grad	Follow prediction.grad_fn back to parameters
NaNs or infinities	Covariance validity and scale explosion	Check finite tensors per stage; lower learning rate after fixing math
Loss falls but object is poor	Background dominates MSE	Inspect a common object crop and unused views
Training-view loss jumps	Camera changes each update	Evaluate a fixed view or mean over all training views
GPU memory error	Dense [N,P] intermediates	Reduce N or size, release old graphs; benchmark actual configuration
CUDA unavailable	Wrong torch wheel or template	Confirm nvidia-smi, then GPU-compatible PyTorch
File not found	Working directory or nested ZIP folder	Use pwd/ls remotely or Get-Location/dir locally

WHEN STUCK / A disciplined debugging order

Position first, covariance second, pixel influence third, blending fourth, gradients fifth, optimization last. Use one or two Gaussians until the failure is understandable. Changing five settings simultaneously makes the result harder to explain.

CHECKPOINT / Before asking for help

State the command, expected behavior, actual behavior, tensor shapes, and smallest failing example. Paste the relevant function and error; avoid dumping the entire project. Ask for a hint about the first failing step before requesting a replacement implementation.

19 / Understand the work you left out

The original 3DGS system combines an explicit Gaussian scene representation, optimization with adaptive density control, and an efficient visibility-aware rasterizer. Its training also learns view-dependent appearance. Our project studies a small subset deeply. Original paper, especially sections 4-6

Full-system component	Our choice and consequence
Structure-from-Motion / camera estimation	Known synthetic cameras. We do not infer poses from photographs.
Initialization from scene evidence	Known count and perturbed target scene. Recovery is easier than reconstruction from scratch.
Densification	Fixed count. We cannot add primitives where representation is insufficient.
Pruning	None. Weak or unnecessary Gaussians remain.
Spherical harmonics	Constant RGB. We cannot model color changes with viewing direction.
Fast GPU rasterizer / backward	Dense PyTorch training and bounded forward exercise. Limited scale; no real-time claim.
Full image objective	Plain MSE. No claim to reproduce the paper's quality or training objective.

Densification and pruning, in plain terms

During full training, image errors and position gradients help identify where the representation needs more detail. Gaussians can be cloned or split into smaller ones; very weak or unsuitable ones can be removed. The number and placement of primitives are therefore adapted, rather than hoping one fixed initial set is adequate.

How this differs from NeRF

Classic NeRF uses a neural network to predict density and view-dependent color at sampled 3D locations, then combines samples along camera rays. This project stores explicit spatial primitives and projects them onto the image. Both connect a 3D representation to images through differentiable rendering; their representations and computational work differ.

INTERVIEW / Is your project generative AI?
It learns a representation of one observed synthetic scene and renders views of it. It is not a model trained to generate new objects from prompts or to sample a general distribution over scenes. Its relevance is 3D vision, differentiable rendering, and direct parameter optimization.

20 / Practice the rendering questions

Answer each aloud before reading the short answer. If you need the equation, derive it from the coordinate definitions rather than recalling an isolated string of symbols.

What is a 3D Gaussian here?

A smooth spatial influence function centered at a 3D position, shaped by covariance, and carrying color and opacity. The renderer approximates its projected influence with a 2D Gaussian.

Why use scale and rotation?

They directly express positive axis spreads and orientation, and construct a valid covariance. Eigenvalues are squared spreads, not spreads themselves.

Why transform covariance with W on both sides?

Each offset transforms as W d. Its outer product transforms as W d d^T W^T. Covariance averages these outer products.

What is the Jacobian doing?

It converts small 3D offsets around the mean into approximate screen offsets. The depth column matters off-axis because changing depth moves the projection toward or away from the image center.

Why is inverse covariance in the pixel calculation?

It measures distance relative to the Gaussian's spread and orientation. The same geometric offset is less significant along a broad axis than a narrow one.

Why multiply transmittance by (1 - alpha)?

That is the fraction remaining visible behind this splat. A farther color contributes only through the fraction that survived all nearer splats.

Why can ordering be imperfect?

A whole extended Gaussian is assigned one center depth. That is not an exact per-pixel geometric ordering of overlapping 3D volumes.

What would moving the camera change?

Camera-space means and covariances, projected centers and ellipses, and possibly depth order. The stored world-space scene stays the same.

CHECKPOINT / Whiteboard exercise

Draw the pipeline from parameters to final image. Annotate the dimensions at every stage. Then trace two Gaussians through one pixel and compute the resulting color with a nonblack background. Finish by identifying the approximation at projection and the approximation at visibility.

20 continued / Defend the learning experiment

What is being learned without a neural network?

Positions, scale, rotation, opacity, and RGB are adjusted directly. The renderer is the differentiable function; a neural network is not required for gradient-based optimization.

How do gradients reach a position?

Through the projected center and covariance, then pixel influence, then visibility-weighted color, then image loss. Scale and rotation also affect the projected covariance.

Does a good rendered image prove correct 3D geometry?

No. Different parameters can produce similar images, especially from one view. Occluded regions are weakly constrained. More views help, but do not prove unique recovery.

Why might one-view training look good from the front and bad elsewhere?

The objective only rewards agreement with that view. Depth and unseen regions can remain wrong. Our helpful initialization may partially hide this failure, which limits the experiment's strength.

How do you know your gradients are correct?

Explain the central finite-difference check, the chosen parameter, why float64 helps, and why you avoid a sorting or clipping boundary. One passing derivative is evidence, not proof for every parameter and scene.

Why not train the full original system?

The learning question was whether you could derive, implement, check, and optimize through the core image-formation calculations. Fixed count and known cameras isolate that question. Full reconstruction adds major problems you have not implemented; acknowledge the narrower claim.

What would you improve next?

Choose one specific limitation supported by your results: harder initialization, independent target images, added Gaussian capacity, a trained isotropic baseline, or efficient tile-based rendering. Explain what experiment would tell you the change helped.

CHECKPOINT / Prepare your own opening

In 4-5 sentences: state the question you explored; name the functions you implemented; state the supplied inputs and scaffolding; give one measured result; and explain one limitation. If you used the answer key or AI assistance, describe that accurately. Do not claim "from scratch" without defining the boundary.

WHEN STUCK / Rehearse under interruption

Have someone interrupt with "why?", "show me in the code", and "how did you check that?" The goal is to reason from your implementation, not to outguess an interviewer's specialty.

Optional / Load an existing Gaussian PLY

Use this only after the main guide works. A trained Gaussian file is learned scene data, not a plain point cloud or a replacement for your rendering implementation. Large files can overwhelm the dense renderer.

The original implementation commonly stores x/y/z, scale_0..2, rot_0..3, opacity, and f_dc_0..2. Scales and opacity are stored in their unconstrained forms; colors use spherical-harmonic coefficients. Check the producing implementation before decoding another file. Official Gaussian model code

```
from plyfile import PlyData
import numpy as np
v = PlyData.read("scene.ply")["vertex"].data
means = np.stack([v[k] for k in ("x", "y", "z")], axis=1)
log_s = np.stack([v[f"scale_{i}"] for i in range(3)], axis=1)
scales = np.exp(log_s)
quats = np.stack([v[f"rot_{i}"] for i in range(4)], axis=1)
raw_o = np.asarray(v["opacity"], dtype=np.float32)
opacity = torch.tensor(raw_o).sigmoid().numpy()
dc = np.stack([v[f"f_dc_{i}"] for i in range(3)], axis=1)
rgb = np.clip(0.5 + 0.28209479177387814 * dc, 0, 1)
```

Ignoring higher-order color coefficients removes view-dependent appearance. This snippet only decodes parameters; it is not a complete external-scene integration command. Inspect property names, lengths, finite values, and scale ranges before constructing a Scene.

Cameras are the other half of the input

Use the scene's documented camera parameters and convert conventions, or deliberately recenter/rescale a small object and adjust the camera consistently. Our synthetic orbit is not automatically a suitable camera for an arbitrary PLY. Do not silently reuse it and conclude the renderer failed.

WHEN STUCK / Keep the scope honest

If you inspect a small subset, label it as a subset; missing splats create holes. Use the bounded forward path for a small external scene and measure it. Do not allocate a full [millions of Gaussians, all pixels] tensor. Record the scene's source/license and exactly which trained attributes you reused.

Read only what unblocks the next step

Resource	When and what to use
3Blue1Brown: Linear transformations	Before covariance if you cannot explain what a matrix does to axes. The embedded video is sufficient; do not watch an entire course today.
PyTorch autograd basics	Before TODO 5. Focus on requires_grad, backward, accumulation, and no_grad.
Original 3DGS paper	After your first render. Read sections 4-6 for context and compare their scope with yours.
Original project page and video	Optional visual context. The video shows the full method, not the quality or speed promised by this toy project.
RunPod quickstart	At deployment; match the instructions to the interface you see.
RunPod connections	If Jupyter/terminal access is unclear.
RunPod storage / Pod lifecycle	Before stopping or terminating; understand what persists.
PyTorch benchmark guide	If timings are implausibly fast or inconsistent.

The derivations and exercises in this workbook are worked explanations of the implemented toy renderer. The cited paper describes the full research system; the supplied code is an educational implementation, not the authors' optimized code.

Final self-check

[ ] I can explain every TODO function without looking at the answer key.
[ ] I can calculate one projected point and one two-splat pixel by hand.
[ ] I have passing numeric checks and an inspected gradient check.
[ ] I trained all parameter groups through my own implemented functions.
[ ] I inspected several held-out images, not just training loss.
[ ] I recorded an experiment, a limitation, and one real debugging decision.
[ ] I downloaded my code, checkpoint, settings, and final visuals.
[ ] I can distinguish my work, supplied scaffolding, and outside sources.

If an item is incomplete, describe it accurately. A smaller, verified claim is more defensible than a broad claim your own code cannot support.